TP11 Gestión de Calidad

- Dado el siguiente requerimiento para la implementación de una función matemática denominada "*Conjetura de Collatz*".

> La conjetura de Collatz, conocida también como conjetura 3n+1 o conjetura de Ulam (entre otros nombres), fue descripta por el matemático Lothar Collatz en 1937, y aun no se ha resuelto. Sea la siguiente operación, aplicable a cualquier número entero positivo:
>
> - Si el número es par, se divide entre 2.
> - Si el número es impar, se multiplica por 3 y se suma 1.
>
> Ahora, se forma una sucesión mediante la aplicación de esta operación repetidamente, comenzando por cualquier entero positivo, y tomando el resultado de cada paso como la entrada del siguiente. La conjetura dice que siempre alcanzaremos el 1 independientemente del número con el que se comience.

El requerimiento es solicitar y aceptar un número entero positivo cualquiera por teclado, comprobar la conjetura de Collatz según la definición anterior calculando el número de iteraciones que son necesarias para alcanzar el valor 1 e imprimir el número de partida y el número de iteraciones. Por cuestiones prácticas limite a 1999 el entero máximo a validar.

El programa desarrollado para tal propósito es:

```
#Calculando el número de iteraciones del algoritmo de Collatz
def collatz(num):
  iteraciones = 0
  while num != 1:
    if num % 2 == 0:
      num = num // 2
    else:
      num = 3*num + 1
    iteraciones += 1
  return iter
#Comprobando los resultados
i=20
print("El número de iteraciones para %d es %d\n" % (i,collatz(j))
```

- Lea con atención las consignas antes de continuar con el ejercicio.
- Estructure los requerimientos funcionales y no funcionales de éste problema. Identifique a los primeros como $R=\{R_1,\ldots,R_n\}$ y a los segundos como $F=\{F_1,\ldots,F_m\}$

- Formule tantos test cases como crea necesario denominando a los mismos $T=\{T_1,\ldots,T_n\}$, indique para cada uno si se trata de un test unitario (caja blanca) o funcional (caja negra).
- Establezca una matriz de trazabilidad RTMX entre $R \cup F \Leftrightarrow T$, valide la integridad del conjunto de prueba basado en la misma.
- Realice (de ser posible en grupo) un ejercicio de inspección, **sin ejecutar el código** trate de capturar tantos defectos latentes sintácticos y semánticos como le sea posible. Registre el número detectado y considere a todo el ejercicio como sesión 1. Considere como defecto una tasa de comentarios mayor a 12.
- Realice (de ser posible) un ejercicio de inspección validando los casos de test desarrollados previamente e intentando agregar nuevos casos. Actualice la matriz de trazabilidad toda vez que un nuevo caso de test sea agregado. Registre el número de defectos detectados y considere a todo el ejercicio como sesión 2.
- Ejecute todos los test cases indicados como de **tipo unitario**, utilice "sesiones" $\{S_1|S_2|S_3..,S_n\}$ para ello donde en cada sesión haga tantos casos de test como le sea posible hasta que no pueda proseguir. Cada intento de ejecución fallido contará como 1 defecto aunque no ocurra asociado a ningún caso de test. Registre cuantos defectos encuentra en cada sesión. Entre sesión y sesión realice todas las reparaciones en el código necesarias para solucionar las fallas observadas durante la última sesión. No progrese a la sesión siguiente sin haber realizado correcciones de todas las fallas observadas en la anterior, con espíritu académico no haga corridas o sesiones individuales para cada falla.
- Cuando todos los casos de test unitarios ejecuten en forma exitosa repita el procedimiento con los casos de test de **tipo funcional**. Proceda en forma de sesiones también en éste caso utilizando como numeración $\{S_{n+1},..,S_m\}$, repita en cada sesión todos los casos de prueba realizados en la previa (exitosos o no) hasta que el 100% de las pruebas realizadas sean exitosas en una dada sesión. Registre para cada sesión el número de fallas observadas.
- Al finalizar el ejercicio debe verificar las siguientes condiciones:
    - Todos los casos de prueba fueron exitosamente ejecutados al menos una vez.
    - No hubo caso de test que no se hubiera ejecutado.
    - No hay requerimiento (funcional o no funcional) que no se haya verificado.
- Obtenga un listado donde se exponga el número total de sesiones realizadas con los defectos encontradas en cada una.

- En el ejercicio anterior considere que cada agrupamiento del test realizado (inspección, unitario, funcional) es una "fase", calcule al finalizar el PCE en cada etapa, a los efectos del ejercicio 100% corresponderá al total de defectos observados.
- Calcule la densidad de defectos del programa al comienzo del test considerando que el programa original tiene *S=18 Locs*.
- Lea el tutorial "*Kanban para uso genérico*" del Ing. A. Ruiz de Mendarozqueta, explique en sus palabras el significado del parámetro TEP de esa metodología y cuál es el significado crítico en su utilización.
- Utilice los algoritmos discutidos en el taller denominado "*taller_defectos.ipynb*" sobre los posibles criterios para evaluar la proyección en los defectos aplicando un modelo de regresión aproximado como el allí mencionado. A continuación, utilice un modelo similar para estimar el número total de defectos a esperar en un aplicativo a partir de los siguientes datos obtenidos en los primeros días de test.

Dia	Defectos
0	9
1	18
2	5
3	7
4	23
5	2
6	8

- Repita el ejercicio anterior pero tomando el dataset de resultados de prueba obtenido en ejercicios anteriores, asimile "Día" a "Sesiones". Estime el número total de defectos de la implementación. Calcule el tamaño final del código (en Locs) usando "*cat collatz.py | wc -l*" o su equivalente en otros sistemas operativos, en función del número de defectos total proyectado estadísticamente (μ_0) en el cálculo y teniendo en cuenta que puede calcular la cantidad de defectos "liberados" al finalizar el test como $\mu_r=\mu_0-\Sigma\mu_i$ siendo μ_i los defectos identificados en las sucesivas etapas de test.
- Lea el artículo adjunto "*Marco para evaluar garantía en desarrollo de software*" y a continuación explique cuál será el comportamiento cualitativo de incrementar el PCE de un proceso de calidad de software en el tiempo total de test óptimo para poder otorgar garantía a ese software.
- Una organización ha establecido una línea de base organizacional sobre su performance de calidad que le permite confiar en entregar sus proyectos con un PCE=89% y defectos al momento del liberación (δ_r) de 0.12 def/FP. Se acaba de finalizar la construcción de un proyecto cuyo tamaño (S) es de 100 FP.
    a. ¿Cuál es la expectativa de defectos totales (μ_0) para ese proyecto?

b. ¿Qué densidad de defectos deberá esperarse al momento de finalizar la construcción (δ_0)?
c. ¿Cuantos defectos debería planear detectar durante el período de V&V para ese proyecto?
d. Si esa misma organización detecta en sus primeros tres días de V&V 20 defectos cada día (60 defectos en total). Fundamente una opinión sobre ¿Cuál cree que será el comportamiento del total de defectos respecto a lo esperable en función de los parámetros históricos?.

- Desarrolle una herramienta en lenguaje Python para convertir archivos de formato JSON a formato CSV. El programa deberá implementarse con una estrategia de filtro.
  El formato de ejecución será:
  ```
  python json2csv.py {-i input.txt} {-o output.txt} {-d delimitador}
  ```
  Si no se informa el argumento "-i" el programa podrá aceptar el archivo de entrada mediante entrada estándar, al mismo tiempo si no se informa el argumento "-o" se emitirá el texto resultante mediante salida estándar. Se utilizará la coma como delimitador, excepto que se indique uno distinto con el argumento "-d"
- Asegure la implementación de controles robustos para las condiciones de excepción que pueda encontrar durante la ejecución. Implemente el proyecto en GitHub utilizando un marco para integración continua con las siguientes características:
  a. Todo el desarrollo lo realizará utilizando un ambiente virtual Python (venv).
  b. Esqueleto de proyecto basado en CookieCutter
  c. En el caso de utilizar herramientas de IA documentará los prompts utilizados en el archivo CONTEXT.md
  d. La naturaleza del proyecto se mantendrá actualizada en un archivo README.md
  e. Los cambios se registrarán en un archivo llamado CHANGELOG.md
  f. Genere un archivo REQUIREMENTS.TXT para listar todas las librerías que pueda necesitar el proyecto para funcionar de forma de generarlas como parte de un ambiente virtual Python.
  g. La licencia del proyecto será MIT y se generará documentación automáticamente con pdoc o sphynx
  h. Se llevará registro de VERSION y BUILD
  i. Ejecutar ruff para validar las reglas de formato y que no de errores.
  j. Ejecutar black para validar el formato consistente.
  k. Utilizar y validar el correcto uso de reglas de formateo PEP8.
  l. Utilizar el correcto uso de convenciones PEP257 para los docstrings, aceptar solo si no hay errores.
  m. Ejecutar MyPy para los modulos dentro de alcance, aceptar solo si no hay errores.
  n. Ejecutar PyRight para los modulos dentro de alcance, aceptar solo si no hay errores.

o. Test unitario con Pytest con hipótesis de test unitario que permitan la cobertura del 85% o mejor.
p. Utilizar bandit para la evaluación básica de seguridad y que no queden observaciones.
q. No utilizar Trufflehog en los controles de seguridad.
r. Producir una documentación básica del funcionamiento del módulos y actualizarla con cada PR exitoso.
s. Actualizar archivo requirements.txt que se usará para las dependencias de librerías.
t. Se automatizará un workflow en github integrando la fase de CI/CD completa.
u. Con cada PR exitoso genera o actualiza la focumentación básica con pdoc o sphinx.

Opcionalmente puede utilizar herramientas de IA para desplegar el proyecto.

- En la presentación "*Agile and software engineering , an invisible bond*" resumen del artículo del mismo nombre se hace referencia la importancia de la deuda técnica como factor de deterioro en la performance de valor de un proyecto, indique de acuerdo al artículo que factores gestionados desde el proceso de calidad mejoran la gestión de deuda técnica y porque producen esa mejora.
- Utilizando el modelo simple de garantía expuesto en las clases teóricas razone como afecta la aplicación de una multa al punto de equilibrio del proceso, fundamente.